

UNIVERSIDAD
DE ANTIOQUIA
1 8 0 3

DESAFÍO II

UdeABook

Red social por consola en C++ (POO)

Informe final

Integrantes

Luis Miguel Cohen Teherán

David Aguirre

Repositorio público: https://github.com/luisCT1/UdeABook-POO-informaticaII

Video de demostración: enlace entregado en Ude@

curso: Informática II

Semestre 2026-2

Universidad de Antioquia

Facultad de Ingeniería

Octubre 2026

Introducción

En este informe presentamos el desarrollo de UdeABook, una red social que funciona desde la consola. El programa permite cargar usuarios desde archivos de texto, iniciar sesión, consultar fotos de amigos, enviar solicitudes, visitar perfiles y álbumes, escribir mensajes y revisar sugerencias de amistad. Al terminar cada operación, también muestra cuántas iteraciones se realizaron y cuánta memoria está utilizando.

El proyecto se desarrolló en C++ aplicando programación orientada a objetos. Por las condiciones del desafío, no usamos herencia entre las clases del negocio, ni contenedores std::vector o std::string, ni estructuras struct. La memoria dinámica se administra con new y delete, y las cadenas se trabajan con char*. El código está distribuido en archivos .h y .cpp; la versión integrada se encuentra en el repositorio público.

Objetivo general

Desarrollar una red social por consola que permita iniciar sesión, gestionar amistades, consultar perfiles, recibir sugerencias y enviar mensajes. El sistema debe medir el uso de memoria y las iteraciones, y funcionar con bases de datos de aproximadamente 1.000 y 5.000 usuarios.

Objetivos específicos

- Diseñar un diagrama de clases que represente los usuarios, las amistades, los mensajes, los álbumes y las fotos, y usarlo como guía para implementar el sistema.
- Guardar los datos en archivos .txt, cargarlos al iniciar el programa y registrar los cambios posteriores sin tener que reescribir todos los archivos.
- Lograr que las consultas de amistad, las sugerencias y la selección de fotos funcionen de manera razonable tanto con 1.000 usuarios, como en la demostración, como con 5.000.
- Mostrar al terminar cada opción del menú un reporte de iteraciones y memoria que corresponda al trabajo realizado.

a. Análisis del problema y alternativa de solución

El reto consistía en construir una red social en la que cada persona tuviera un nombre de usuario, una contraseña y datos de perfil. También debía poder gestionar amistades y solicitudes, intercambiar mensajes de hasta 512 bytes y organizar fotos en álbumes. Como el programa funciona por consola, consultar una foto significa mostrar su ruta de archivo, no abrir la imagen.

Desde el comienzo tuvimos que considerar varias condiciones del enunciado:

- La información debía cargarse y actualizarse desde archivos, sin convertir la carga en una opción del menú para el usuario.
- Al iniciar sesión, el programa debía mostrar tres fotos de perfil de amigos y permitir pedir otro grupo distinto las veces que se quiera, sin repetir el trío que ya está en pantalla.
- El envío y la aceptación de solicitudes debían validar que el destinatario existiera, que no fuera el mismo usuario, que no hubiera una amistad previa y que no existiera otra solicitud pendiente.
- Al visitar el perfil de un amigo, el sistema debía respetar la privacidad de sus álbumes y permitir dar 'me gusta' a las fotos visibles.
- Las sugerencias debían basarse en los amigos en común y mostrar entre una y veinte personas.
- El módulo de mensajes debía permitir enviar mensajes, consultar los nuevos y buscar conversaciones dentro de un rango de fechas. Para la demostración usamos un intervalo de varias semanas.

- Al terminar cada funcionalidad, debíamos reportar las iteraciones realizadas, incluyendo las de funciones internas, operaciones con archivos y procesos aleatorios, junto con la memoria utilizada en ese momento.

También debíamos respetar las restricciones de implementación: no usar contenedores de la STL ni struct, administrar la memoria dinámica manualmente, trabajar con varios archivos fuente y evitar copias innecesarias de objetos grandes. El proyecto sí permite sobrecargar operadores y usar plantillas, por lo que aprovechamos esas herramientas donde resultaban útiles.

Qué descartamos y qué nos quedó

Al principio consideramos representar las amistades con una matriz de valores bool. Sin embargo, con 5.000 usuarios ocuparía bastante memoria y recorrer filas completas repetidamente sería costoso. También evaluamos usar únicamente listas, pero consultar si dos personas son amigas resultaría más lento. Finalmente combinamos ambas alternativas: una matriz de bits para hacer consultas rápidas y listas de adyacencia para recorrer solo las amistades existentes.

Como no podíamos utilizar std::vector, implementamos nuestra propia estructura ListaDinamica<T>. También dividimos el código en varios archivos para facilitar el trabajo en equipo. La clase RedSocial funciona como punto central del sistema, de modo que el menú no necesita interactuar directamente con cada componente.

Para guardar los cambios, descartamos reescribir archivos completos cada vez que alguien da 'me gusta' o envía un mensaje. Esa estrategia habría sido más lenta y propensa a errores. En su lugar, cargamos los datos al inicio y después agregamos las nuevas líneas o registramos los cambios en una bitácora.

Cómo quedó armado el sistema

Organizamos el sistema en tres grupos de clases para mantener el código ordenado:

- Soporte: FechaHora y ListaDinamica<T>, utilizadas por buena parte del sistema.
- Dominio: Usuario, Mensaje, AlbumFotos y Foto.
- Sistema: RedAmistades, GestorArchivos, MedidorRecursos y RedSocial.

Una sesión comienza cargando por defecto la base mediana (unos 1.000 usuarios). Si se necesita otra base, se cambia la ruta a mano en main.cpp (pequeña o grande). Tras una carga correcta el programa avisa que fue exitosa, se crea RedSocial, se leen los seis archivos y se pide el inicio de sesión. Desde el menú se puede entrar a amistades (incluyendo ver la lista de amigos), perfiles, sugerencias y mensajes, consultar otras fotos, cerrar sesión o salir. Los cambios importantes se guardan a medida que ocurren, y al terminar cada opción se presenta el reporte de recursos.

Para mejorar el rendimiento, tomamos varias decisiones:

- Los usuarios se almacenan en un arreglo compacto y ordenado por nombre, lo que permite usar búsqueda binaria durante el inicio de sesión.
- Las amistades y las solicitudes se consultan mediante bits, con un costo cercano a O(1) para comprobar si existe una relación.
- Las listas de amigos permiten recorrer únicamente las conexiones existentes, sin revisar toda una fila de la matriz.
- Los amigos en común se calculan aplicando AND entre palabras de 64 bits.

- El índice fotosPorId permite actualizar los 'me gusta' de una foto sin volver a recorrer todo el archivo multimedia.
- La bandeja de mensajes se mantiene ordenada para localizar mensajes por fecha mediante búsqueda binaria.
- Cuando medimos memoria, pausamos temporalmente el conteo de iteraciones para que el propio cálculo no altere el resultado.

Formato de los archivos (cómo guardamos la red)

Cada conjunto de datos se guarda en su propia carpeta: `datos/pequenio`, `datos/mediano` y `datos/grande`. En cada una hay seis archivos `.txt` con campos separados por `|`. Las fechas usan 14 dígitos (`AAAAMMDDhhmmss`). Las rutas de las imágenes se construyen cuando se necesitan a partir de una subruta fija común, por lo que no se guardan completas en los archivos. La carga sigue este orden: usuarios, multimedia, amistades, solicitudes, mensajes y cambios. Si ocurre un error crítico, el proceso se detiene para evitar que el sistema quede cargado solo parcialmente.

usuarios.txt

Cada línea representa a una persona. Su posición, empezando desde cero, se usa como índice en memoria y en las matrices. Por eso, el archivo debe estar ordenado por nombre de usuario.

nombreUsuario|fechaNacimiento|correo|contraseña|ciudad|país|fechaInscripcion

Ejemplo:
alejandro_betancur038|20021221101330|alejandro_betancur038@udea.edu.co|clave123|Cali|Colombia|20241128203105

multimedia.txt

El archivo contiene líneas de álbumes (A) y de fotos (F). Cada foto pertenece al último álbum registrado antes de ella. La privacidad se indica con P para público, R para privado y A para visible solo para amigos.

A|indiceUsuario|nombreAlbum|privacidad
F|fechaCaptura|tamanoBytes|nombreArchivo

Ejemplo:
A|1|viaje_cartagena|A
F|20261004112303|2787|viaje_cartagena_1
F|20260603205210|4088|viaje_cartagena_2

Al mostrar una foto, el programa construye una ruta como `/users/<subRutaFija>/<usuario>/albums/<album>/<archivo>.png`. Para la foto de perfil utiliza la ruta `.../albums/profile.png`.

amistades.txt y solicitudes.txt

Estos archivos guardan pares de índices de usuarios. Por ejemplo, `0|1` puede representar una amistad y `78|86` una solicitud. Si al cargar los datos se encuentra una solicitud entre personas que ya son amigas, se elimina esa solicitud. Cuando se acepta una solicitud, se actualiza la relación y se agrega la amistad al archivo.

mensajes.txt

Cada línea registra el emisor, el receptor, la fecha y hora, la longitud del texto en bytes y el contenido del mensaje. El texto se lee usando la longitud indicada, no simplemente hasta encontrar el final de la línea.

cambios.txt

Este archivo conserva cambios puntuales para evitar reescribir archivos grandes:

- L|usuario|primerNoLeido — hasta dónde ya se marcaron leídos.
- K|idFoto|likes — likes actualizados.

Preparamos tres bases de datos para probar el comportamiento del programa: la pequeña, con cerca de 100 usuarios; la mediana, con unos 1.000, usada en el video; y la grande, con aproximadamente 5.000. En la base mediana usamos `alejandro_alvarez152` con la contraseña `clave123` para la demostración. También incluimos usuarios con varios álbumes, como `alejandro_alvarez722`, para mostrar la consulta de perfiles.

b. Diagrama de clases

Incluimos dos versiones del mismo diagrama de clases: una vista reducida para entender la arquitectura general y otra ampliada con los atributos y métodos. El menú no se modeló como una clase, ya que sus funciones se limitan a llamar a RedSocial.

RedSocial contiene la lista de usuarios y coordina los componentes RedAmistades, GestorArchivos y MedidorRecursos. Cada Usuario es responsable de sus álbumes y de los mensajes que recibe; los mensajes enviados se referencian desde el usuario correspondiente. AlbumFotos contiene sus objetos Foto. FechaHora y ListaDinamica se reutilizan en distintas partes del proyecto. La relación de amistad se administra en RedAmistades, no dentro de Usuario.

Operadores que sobrecargamos: < y == en Usuario y FechaHora, << amigo en FechaHora, [] y = en ListaDinamica, ++/-- en Foto (likes), = en RedAmistades, < en Mensaje por fecha.

Vista reducida

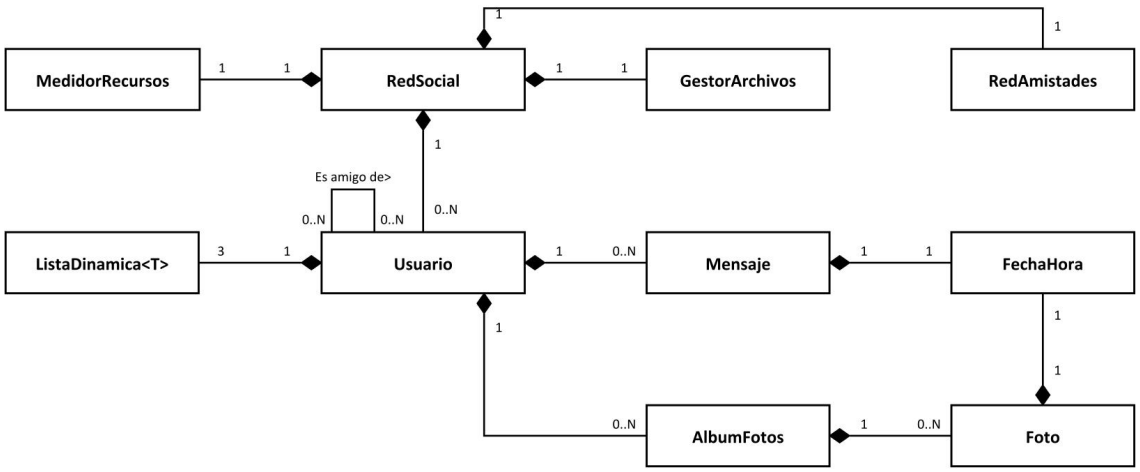


Figura 1. Diagrama de clases reducido. diagrama_Clase_resumen.svg

Vista ampliada

Si el diagrama ampliado no se alcanza a leer bien en Word, también se encuentra el archivo SVG `diagrama_Clase_completo.svg` en la carpeta del proyecto.

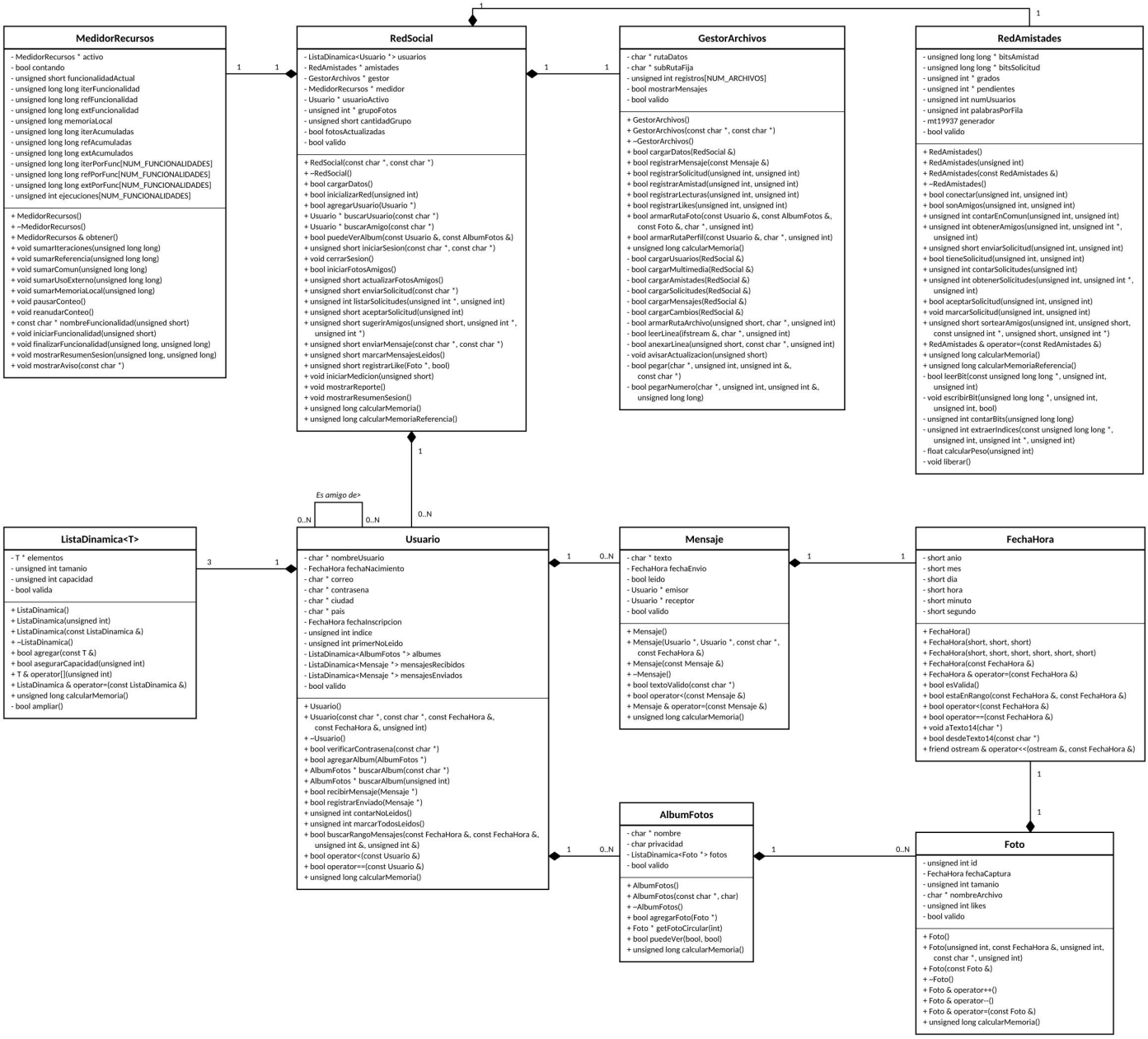


Figura 2. Diagrama de clases ampliado. diagrama_Clase_completo.svg.

Las diez clases resumidas

Clase	Para qué la usamos
FechaHora	Fechas con hora, validarlas y compararlas
ListaDinamica<T>	Nuestro “vector”: arreglo que crece solo
Usuario	Una persona, sus álbumes y sus bandejas
Mensaje	Texto de hasta 512 bytes entre dos usuarios
AlbumFotos	Álbum con privacidad y lista de fotos
Foto	Nombre, fecha, tamaño y likes

RedAmistades	Quién es amigo / quién pidió amistad
GestorArchivos	Lee y anexa los 6 txt; arma rutas
MedidorRecursos	Cuenta iteraciones y memoria
RedSocial	Fachada: el menú solo habla con ella

Especificaciones de cada clase:

FechaHora: valida las fechas, incluidos los años bisiestos, y permite compararlas. Para buscar mensajes por intervalos se compara el día. La fecha se almacena con 14 dígitos.

ListaDinamica: es una plantilla que administra el tamaño y la capacidad de un arreglo. Cuando se llena, duplica su capacidad. Para evitar copias innecesarias, normalmente almacena punteros a los objetos. La responsabilidad de crear y destruir cada objeto está definida, y calcularMemoria cuenta la capacidad reservada.

Usuario: almacena el nombre, correo, contraseña, ciudad, país, fechas y el índice asignado. También administra sus álbumes y los mensajes recibidos; los enviados solo se referencian. El atributo primerNoLeido permite saber desde dónde hay mensajes nuevos. Los amigos se gestionan en otra clase y los usuarios no se copian.

Mensaje: contiene el emisor, el receptor, la fecha, el texto y el estado de lectura. RedSocial valida la longitud del texto antes de crear el mensaje.

AlbumFotos y Foto: administran los álbumes, su privacidad y las fotos. El carrusel permite recorrer las imágenes de forma circular y los operadores ++ y -- actualizan los 'me gusta'. La ruta completa de cada imagen la construye el gestor de archivos.

RedAmistades: administra las amistades y solicitudes mediante dos matrices de bits, junto con los grados y las listas de adyacencia. Incluye operaciones para conectar usuarios, enviar y aceptar solicitudes, contar amigos en común y seleccionar amigos al azar. La selección usa pesos de 1, 2.5 o 3.5 según el número de amigos compartidos, con umbrales de 10 y 20, y utiliza mt19937.

GestorArchivos: concentra las operaciones de lectura y escritura, además de mantener el índice de fotos. Así, RedSocial no necesita conocer los detalles del formato de los archivos `.txt`.

MedidorRecursos: mantiene contadores estáticos que las clases utilizan para registrar iteraciones, operaciones externas y memoria local. Al finalizar cada funcionalidad, genera el reporte correspondiente.

RedSocial: coordina el inicio de sesión, las fotos, las solicitudes, las sugerencias, los mensajes, los 'me gusta' y los reportes. Es la interfaz que utiliza el menú para acceder a la lógica del sistema.

c. Algoritmos esenciales (en alto nivel)

En esta sección describimos la lógica de los algoritmos principales sin incluir el código. Para analizar su costo, tomamos como referencia la base de unos 1.000 usuarios utilizada en la demostración.

Carga

Primero cargamos los usuarios en un arreglo compacto y ordenado, y creamos la estructura de la red. Después se leen los álbumes y las fotos, se construye el índice por identificador y se cargan las amistades, las solicitudes y los mensajes en las bandejas correspondientes. Por último, se aplican los cambios guardados en cambios.txt. Si falla una parte crítica, se cancela la carga. Aunque este proceso no aparece en el menú, también medimos los recursos que consume.

Login y las 3 fotos

Para iniciar sesión, buscamos el nombre de usuario mediante búsqueda binaria, verificamos la contraseña y activamos la cuenta. Luego seleccionamos tres amigos y asignamos a cada uno un peso según la cantidad de amigos en común. Se sortean tres personas distintas y se muestran sus fotos de perfil. El usuario puede pedir otro grupo las veces que quiera; en cada refresco se excluye el trío que estaba en pantalla para no repetirlo de inmediato.

Amigos en común

Para contar los amigos en común, aplicamos AND a las filas de bits de ambos usuarios y contamos los bits activos con un ciclo propio. Con 1.000 usuarios se procesan alrededor de 16 palabras por fila, por lo que la operación resulta económica para cada pareja. Este cálculo también se utiliza en las sugerencias y en la selección de fotos.

Solicitudes

Al enviar una solicitud, verificamos que el destinatario exista, que no sea el usuario actual, que no haya una amistad previa y que no exista otra solicitud pendiente. Si todo está bien, registramos la solicitud en memoria y en el archivo. Para aceptar una, mostramos las solicitudes recibidas, creamos la amistad, retiramos la solicitud pendiente y guardamos el cambio. Si una validación falla, el sistema devuelve un código para que el menú explique el motivo.

Sugerencias

Para generar sugerencias, revisamos los posibles candidatos y descartamos a quienes ya son amigos, tienen una solicitud pendiente o no comparten amigos con el usuario actual. De los restantes conservamos los mejores en una lista de hasta 20 personas, ordenada por inserción. Así evitamos ordenar la lista completa de usuarios. Desde el menú se puede enviar una solicitud a cualquiera de las personas sugeridas.

Perfil, privacidad y carrusel

El perfil de otro usuario solo se puede consultar si existe una amistad. Se muestran los álbumes permitidos: los públicos siempre están disponibles, los de tipo A se muestran a los amigos y los privados no aparecen durante la visita. El carrusel recorre las fotos de forma circular. Cuando cambia la cantidad de 'me gusta', el programa actualiza el archivo de cambios.

Mensajes

Los mensajes solo se pueden enviar a amigos. El texto no puede estar vacío ni superar los 512 bytes, y la fecha se registra en hora local (UTC-5). El mensaje se agrega a las bandejas del emisor y del receptor y se guarda en el archivo. Al consultar los nuevos, el sistema empieza en primerNoLeido, marca los mensajes como leídos y registra el avance. Para buscar por fechas, usa dos búsquedas binarias sobre la bandeja ordenada y muestra los mensajes del intervalo solicitado.

Medición

Al entrar a una opción, reiniciamos los contadores. Las iteraciones se suman durante los ciclos; las operaciones con archivos y los procesos aleatorios se registran como uso externo, y también se contabiliza la memoria de los buffers locales. Al terminar, se calcula el consumo de las estructuras y de la memoria local para mostrar el reporte. También se presenta un resumen al cerrar sesión o salir del programa.

d. Problemas que nos salieron

Durante el desarrollo aparecieron varios problemas que tuvimos que resolver:

Administración de memoria. En las primeras versiones algunos mensajes se liberaban dos veces porque tanto el emisor como el receptor parecían ser sus dueños. También olvidábamos liberar las listas de adyacencia al destruir RedAmistades. Para corregirlo, definimos un único responsable de cada objeto: Usuario administra los mensajes recibidos y los álbumes, mientras que RedSocial es responsable de los usuarios y los componentes principales.

Carga incompleta. Algunas versiones dejaban posiciones nullptr o mantenían una red inconsistente cuando un archivo fallaba a mitad de la lectura. Cambiamos a un arreglo compacto y establecimos que la carga se detuviera si el formato no era válido. El índice de fotos también surgió como respuesta a estos problemas.

Consultas y sugerencias lentas. Con 5.000 usuarios, recorrer únicamente los bits resultaba costoso en operaciones frecuentes, como consultar amigos o calcular grados. Por eso incorporamos listas de adyacencia al registrar las amistades y usamos una lista top-K para conservar las mejores sugerencias sin ordenar todos los candidatos.

Datos poco útiles para la demostración. Una de las bases generadas concentraba casi todos los mensajes en un mismo día, lo que impedía demostrar búsquedas de varias semanas. También había contraseñas repetidas. Generamos de nuevo las tres bases, distribuimos mejor las fechas, añadimos más álbumes a los perfiles que usaríamos en el video y asignamos contraseñas únicas. Esto tomó tiempo, pero permitió mostrar mejor las funciones del programa.

Medición de recursos. Al calcular la memoria también se estaban contando iteraciones, lo que alteraba los reportes. Para evitarlo, incorporamos una opción para pausar y reanudar el conteo y separamos las iteraciones propias de la operación, las de referencia y los usos externos.

Git y Drive. Al mantener el repositorio dentro de 'Mi unidad', aparecieron archivos desktop.ini que interferían con algunos procesos de Git. Lo resolvimos con .gitignore y, cuando fue necesario, trabajamos fuera de Drive. Al integrar las ramas también tuvimos que resolver diferencias entre el menú y el medidor de recursos. Finalmente dejamos que el menú se comunicara solo con RedSocial y conservamos la función pausarConteo.

Cambio de usuario. Al principio era necesario cerrar el programa para iniciar sesión con otra cuenta. Añadimos la opción de cerrar sesión y volver al login sin cargar nuevamente toda la base de datos, lo que también facilitó la grabación del video.

e. Cómo fue evolucionando e implementación

Comenzamos revisando el enunciado y definiendo las clases que necesitábamos. Luego elaboramos el diagrama y un documento interno para repartir el trabajo sin duplicar esfuerzos. Después implementamos la estructura básica: ListaDinamica, FechaHora, las clases del dominio, el gestor de archivos y una primera carga de datos. Cuando esa parte estuvo estable, agregamos RedAmistades, el inicio de sesión y el menú. Más adelante incorporamos solicitudes, sugerencias, perfiles y mensajes, y conectamos el medidor de recursos con cada opción. En la etapa final mejoramos el rendimiento, generamos de nuevo los datos de prueba, añadimos el cierre de sesión e integramos todo en main.

Durante la implementación tuvimos presentes varios aspectos:

- El proyecto se abre en Qt Creator mediante `Desafio_2.pro`; la plantilla de ListaDinamica está en `ListaDinamica.tpp`.
- Los datos se mantienen en memoria mientras se ejecuta el programa. El disco se utiliza para cargar la información al inicio y guardar los cambios posteriores.
- Las cadenas se manejan con char* y funciones propias. El límite de 512 para los mensajes se mide en bytes, por lo que caracteres como la ñ pueden ocupar más de uno.
- El índice de cada usuario coincide con su fila y columna en las matrices y con su posición en usuarios.txt. Si el generador altera ese orden, el sistema puede fallar.
- No implementamos la opción de rechazar solicitudes porque no estaba incluida entre las opciones solicitadas para el menú.
- Al visitar el perfil de un amigo, los álbumes privados no se muestran; sí aparecen los públicos y los habilitados para amigos.
- El usuario puede pedir otro grupo de fotos de amigos las veces que quiera; en cada refresco se excluye el trío que estaba en pantalla para no repetirlo de inmediato.

Menú como quedó

Después de iniciar sesión, el menú ofrece estas opciones: 1) amigos y solicitudes (agregar, ver pendientes y ver mis amigos), 2) ver el perfil de un amigo, 3) personas que quizá conozcas, 4) mensajes, n) consultar otras fotos de amigos, c) cerrar sesión y s) salir. Dentro de cada sección se pueden agregar o aceptar solicitudes, listar amigos, escribir y consultar mensajes, buscar por fechas y recorrer fotos para dar me gusta. Al terminar cada bloque se muestra el reporte de recursos.

Conclusión

Al terminar el proyecto conseguimos una estructura centralizada en RedSocial, una representación de amistades que combina bits y listas, un sistema de archivos sencillo y una medición de recursos que podemos explicar. La solución cumple con los requisitos planteados y nos permitió dividir el trabajo entre los dos. Tanto la base de 1.000 usuarios como la de 5.000 funcionan con el mismo diseño de clases.